Music Genre Classification using Deep Learning

COMP6252 Coursework 1 – University of Southampton

Akash Abbigeri
ECS User ID: 37603035
aa1g25@soton.ac.uk

1 Introduction

Six neural network classifiers are implemented and evaluated on the GTZAN music genre dataset [1], covering ten genres. Net1–Net4 operate on MEL spectrogram images; Net5–Net6 operate on pre-extracted audio features. All models use PyTorch with a 70/20/10 train/val/test split (seed 42) and batch size 32.

2 Dataset and Preprocessing

GTZAN contains 1,000 audio clips (100 per genre), each 30 seconds long. Spectrogram images (180×180, normalised) are used for Net1–Net4 via `torchvision.datasets.ImageFolder` with `transforms.Resize`. For Net5–Net6, raw `.wav` loading via `librosa` was initially attempted but produced severe overfitting (train: 99.9%, test: 23.8%) due to high-dimensional MFCC sequences relative to the small dataset. The pre-extracted `features_3_sec.csv` was therefore adopted: 57 audio features (MFCCs, chroma, spectral centroid, ZCR, etc.) per 3-second chunk, grouped into sequences of shape $(10, 57)$ per song and standardised with `StandardScaler`.

3 Architectures and Training

Net1 – FC Network: `AdaptiveAvgPool2d(16,16)` reduces input from 97,200 to 768 features before two FC hidden layers (512, 256) with BatchNorm1d, ReLU, Dropout(0.4). Kaiming initialisation applied. Adam, lr=10^{-4}, wd=10^{-4}.

Net2 – CNN: Four conv layers in two blocks (32, 64, 128, 256 channels) with MaxPool. `AdaptiveAvgPool2d(4,4)` reduces FC inputs from 524,288 to 4,096 before Linear(4096,256)→Linear(256,10). Adam, lr=10^{-4}, wd=10^{-4}.

Net3 – CNN+BN: Net2 with `BatchNorm2d` after every conv layer. Same hyperparameters.

Net4 – CNN+BN+RMSProp: Net3 architecture with RMSProp optimiser, lr=10^{-4}, wd=10^{-4}.

Net5 – Bidirectional LSTM: 2-layer BiLSTM (hidden=128) on $(10, 57)$ sequences. Hidden states concatenated → Linear(256,128)→ReLU→Dropout(0.3)→Linear(128,10). Adam lr=10^{-3}, gradient clipping (max norm=1.0), `ReduceLROnPlateau`, early stopping (patience=20).

Net6 – LSTM+GAN: Same LSTM as Net5, trained on GAN-augmented data. A conditional GAN (100 epochs) generates synthetic $(10, 57)$ sequences conditioned on genre label, doubling the training set. Generator: noise+label→FC layers with BatchNorm→Tanh. Discriminator: FC layers with LeakyReLU→Sigmoid. Both trained with Adam lr=2×10^{-4}, betas=(0.5,0.999).

4 Results

Table 1: Test accuracy. Net1–Net4: 50ep / 100ep. Net5–Net6: early stopping.

Model	Arch	Opt	Acc (%)
Net1	FC	Adam	48.51 / 55.45
Net2	CNN	Adam	53.47 / 55.45
Net3	CNN+BN	Adam	68.32 / 72.28
Net4	CNN+BN	RMSProp	65.35 / 74.26
Net5	LSTM	Adam	**85.15**
Net6	LSTM+GAN	Adam	74.26

Table 2: Overfitting analysis. Gap = Train Acc − Val Acc. **Bold** = severe (>20%).

Model	Test	Train	Val	Gap
Net1 (50ep)	48.51	62.52	51.76	10.8
Net1 (100ep)	55.45	80.83	53.77	**27.1**
Net2 (50ep)	53.47	47.78	39.20	8.6
Net2 (100ep)	55.45	57.65	50.75	6.9
Net3 (50ep)	68.32	70.82	60.80	10.0
Net3 (100ep)	72.28	84.84	70.35	14.5
Net4 (50ep)	65.35	69.67	59.80	9.9
Net4 (100ep)	74.26	83.12	68.34	14.8
Net5 (LSTM)	85.15	100.00	81.41	18.6
Net6 (LSTM+GAN)	74.26	95.97	73.87	**22.1**

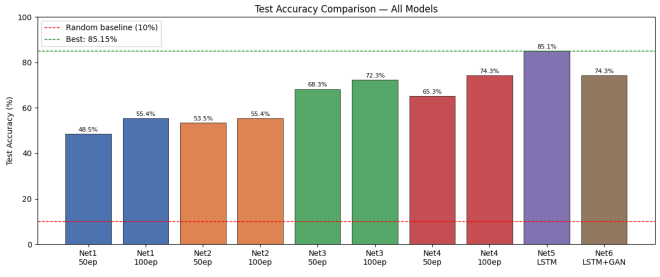

Figure 1: Test accuracy across all models. Red dashed = random baseline (10%). Green dashed = best (Net5, 85.2%).

5 Discussion

FC vs CNN. Net1 achieves 55.5% at 100 epochs but suffers the worst image-model overfitting (gap: 27.1%). FC networks treat pixels independently and lack spatial inductive bias, forcing memorisation over generalisation. Net2 (CNN) reaches 55.5% at 100 epochs with a smaller gap (6.9%) but is limited by gradient instability without batch normalisation.

Batch Normalisation. Adding BN in Net3 yields the largest single architectural gain — 72.3% vs 55.5% for Net2 at 100 epochs. BN reduces internal covariate shift, stabilises gradients, and implicitly regularises training (gap 10.0% at 50 epochs). The gap widens to 14.5% at 100 epochs as the model saturates the training distribution.

Adam vs RMSProp. Net4 (RMSProp) converges slower than Net3 (Adam) at 50 epochs (65.3% vs 68.3%) but surpasses it at 100 epochs (74.3% vs 72.3%). RMSProp's conservative updates avoid over-adaptation in later training, demonstrating that optimiser choice affects not just convergence speed but the final performance ceiling.

Overfitting. Overfitting affects all models due to GTZAN's small training set (700 samples). Net1 (gap 27.1%) and Net6 (gap 22.1%) show the most severe cases. For Net6, GAN-generated data doubles the training set but fails to introduce genuine diversity — the generator reproduces the learned distribution, reinforcing rather than mitigating overfitting. This explains why Net6 (74.3%) underperforms Net5 (85.2%) despite more training data. Regularisation (Dropout, weight decay, gradient clipping, early stopping) kept gaps below 15% for Net2–Net4.

Audio vs MEL Spectrograms. Net5 (85.2%) and Net6 (74.3%) substantially outperform all image-based models. MEL spectrograms encode audio implicitly as pixel intensities ($180 \times 180 \times 3 = 97{,}200$ values); the CNN must learn genre-discriminative properties (rhythm, harmony, timbre) from raw pixels — a hard problem on 700 samples. Pre-extracted CSV features provide 57 semantically rich audio descriptors per chunk, encoding timbral texture (MFCCs), harmonic content (chroma), brightness (spectral centroid), and percussiveness (ZCR) directly. The LSTM receives inputs that already encode genre-relevant structure, explaining the $\sim$25 percentage point advantage.

Overall. Net5 (LSTM, 85.2%) achieves the best accuracy. Among image-based models, Net4 (RMSProp) performs best at 74.3%. Batch normalisation is the single most impactful CNN improvement, and pre-extracted audio features are a more effective modality than MEL spectrograms at this dataset scale.

6 Reflection: Batch Normalisation in Deep Learning

6.1 Why the Topic is Important

Batch normalisation (BN) [2] is among the most impactful advances in deep learning. Before BN, deep networks were highly sensitive to weight initialisation and learning rates — small perturbations in early layers caused large distribution shifts in later layers (*internal covariate shift*). BN normalises inputs within each mini-batch to zero mean and unit variance, applying learnable scale (γ) and shift (β). This enables higher learning rates, reduces initialisation sensitivity, and acts as an implicit regulariser. In this work, Net3 outperforms the identical Net2 by 17 percentage points at 100 epochs, and achieves a tighter overfitting gap (10.0% vs 8.6% at 50 epochs with far higher accuracy), directly demonstrating BN's practical value.

6.2 Current Deep Learning Technologies

Several normalisation variants address BN's limitations. **Batch Normalisation** [2] works well for large-batch CNNs. **Layer Normalisation** normalises across features, preferred for RNNs and Transformers (batch size = 1). **Group Normalisation** normalises within channel groups, robust to small batches. **Instance Normalisation** normalises per sample per channel, used in style transfer. **Spectral Normalisation** bounds weight matrix Lipschitz constants, widely used in GANs.

6.3 Implementation Capability

BN (Net3, Net4) and Dropout have been successfully implemented. Layer Normalisation is understood and could be applied via `nn.LayerNorm` to the LSTM models to reduce their overfitting. Group and Spectral Normalisation present greater challenges — tuning the number of groups or Lipschitz bound requires deeper understanding of gradient dynamics not yet fully developed.

6.4 Positive and Negative Impacts, and Future Vision

Normalisation technologies enabled extremely deep architectures (ResNet, VGG, BERT) and democratised deep learning by reducing the need for expert tuning. However, BN breaks the

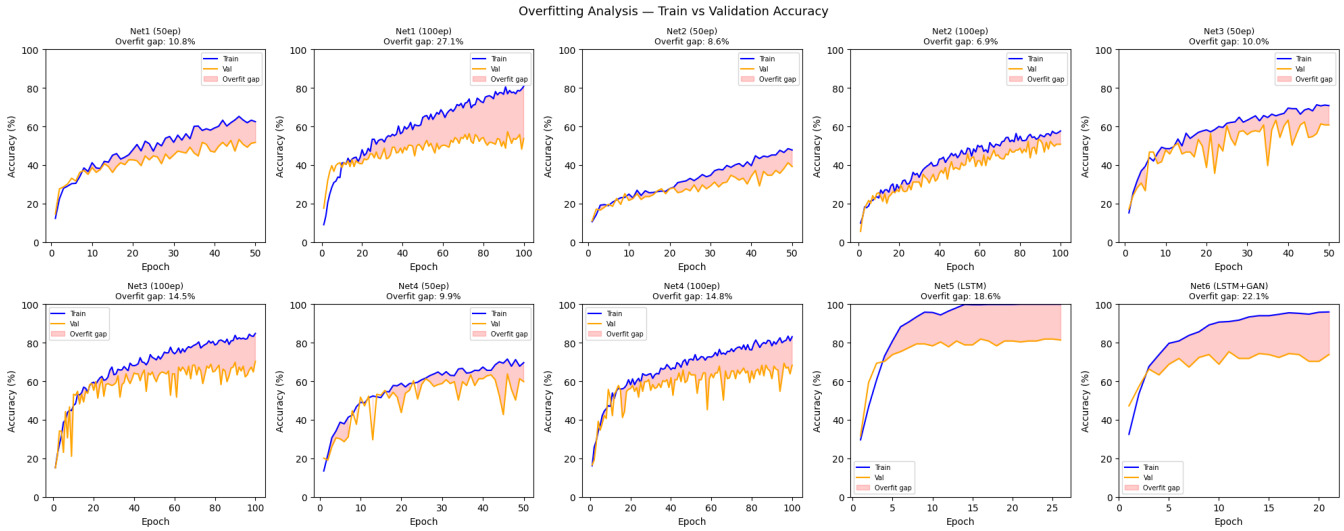

Figure 2: Train vs validation accuracy for all model runs. Red shading indicates the overfitting gap.

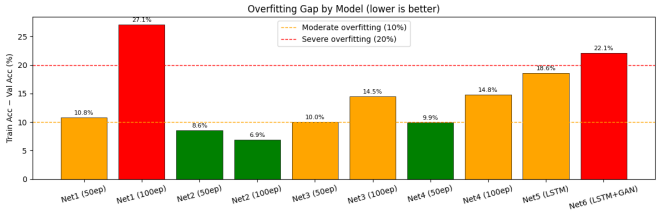

Figure 3: Final overfitting gap per model. Green $<10\%$, orange 10–20%, red $>20\%$.

i.i.d. assumption by creating inter-sample dependencies within batches, causing train/test behaviour differences — harmful on small datasets like GTZAN. Looking forward, adaptive normalisation conditioned on input content (e.g., AdaIN) will become increasingly important in multimodal and generative models. Integration with neural architecture search may eventually automate normalisation design choices that are currently made manually.

References

[1] G. Tzanetakis and P. Cook, *Musical Genre Classification of Audio Signals*, IEEE Transactions on Speech and Audio Processing, 2002. 1

[2] S. Ioffe and C. Szegedy, *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*, ICML, 2015. 2